Scalable Analysis of Vegetation Anomalies Preceding Plant Disease Outbreaks

THRISHA AMBAREESHARAJE URS URS | 862638215, University of California, Riverside, USA
SOHUM DAMANI | 862621529, University of California, Riverside, USA
YASHASWINI DIGGAVI | 862620058, University of California, Riverside, USA
VEDANT BORKUTE | 862552981, University of California, Riverside, USA
SHREYANGSHU BERA | 862485337, University of California, Riverside, USA

ACM Reference Format:
Thrisha Ambareesharaje Urs Urs | 862638215, Sohum Damani | 862621529, Yashaswini Diggavi | 862620058, Vedant Borkute | 862552981, and Shreyangshu Bera | 862485337. 2018. Scalable Analysis of Vegetation Anomalies Preceding Plant Disease Outbreaks. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 15 pages. https://doi.org/XXXXXXX.XXXXXXX

Group Name: Lake it Easy+Pulse Crew

Abstract

Early detection of plant diseases is an important challenge in both agricultural and forested ecosystems, where delayed identification can cause significant ecological and economic damage. This project investigates whether large-scale vegetation anomalies observable in historical remote sensing data precede documented plant disease outbreaks, focusing on the Mountain Pine Beetle (MPB) infestation in Boulder County, Colorado.

We construct a scalable, cloud-native Big Data pipeline that ingests satellite imagery as Cloud Optimized GeoTIFFs (COGs) on Amazon S3, indexes spatiotemporal metadata in a PostGIS database with GIST spatial indexing, and processes pixel-level spectral data using Apache PySpark. Three distinct real-world datasets are analyzed independently through the same pipeline: a 2015–2026 archive at 50 m resolution (4.3 GB, 594 objects), a 2024–2026 archive at 30 m resolution (2.8,GB, 98 objects), and a 2019–2026 archive at 20 m resolution (10.8,GB, 109 images).

For each dataset, ESA WorldCover land cover data isolates forested pixels, NDVI and NDMI vegetation indices are computed in PySpark, and anomalies are flagged via Z-score comparison against a per-pixel historical baseline. Evaluation demonstrates that GIST-indexed queries reduce latency by 15–25× over sequential scans, that PySpark throughput scales approximately linearly with data volume.

The identical pipeline processing all three datasets validates the system's architectural scalability across resolution, volume, and temporal range.

Authors' Contact Information: Thrisha Ambareesharaje Urs Urs | 862638215, turs001@ucr.edu, University of California, Riverside, Riverside, California, USA; Sohum Damani | 862621529, sdama009@ucr.edu, University of California, Riverside, Riverside, California, USA; Yashaswini Diggavi | 862620058, ydigg001@ucr.edu, University of California, Riverside, Riverside, California, USA; Vedant Borkute | 862552981, vbork001@ucr.edu, University of California, Riverside, Riverside, California, USA; Shreyangshu Bera | 862485337, sbera004@ucr.edu, University of California, Riverside, Riverside, California, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
Manuscript submitted to ACM

1 Introduction

Plant diseases have historically caused significant ecological and economic damage. In agricultural regions, disease detection often relies on human observation and localized monitoring. In contrast, forested and remote regions lack continuous surveillance. Advances in satellite imagery and environmental data collection enable long-term observation of vegetation behavior across large geographic regions, but analyzing such data effectively requires approaches that go beyond traditional single-machine scripts and instead follow scalable data processing paradigms [18, 19].

The Mountain Pine Beetle (MPB), *Dendroctonus ponderosae*, provides a compelling case study. Over the past two decades, MPB outbreaks have caused the mortality of hundreds of millions of trees across western North America, profoundly disrupting forest carbon storage, water cycling, and wildfire dynamics. Research has shown that vegetation spectral signatures, particularly NDVI and moisture-sensitive indices, begin to diverge from healthy seasonal patterns weeks to months before visible symptoms appear [9, 13]. Detecting these subtle shifts at scale requires a Big Data infrastructure capable of ingesting, indexing, and querying large volumes of multi-temporal raster data.

The primary objective of this project is to analyze historical vegetation trends in regions where plant diseases have been reported. For selected disease events, vegetation behavior is studied over multiple years before the documented detection date. The analysis seeks to identify deviations or anomalies in vegetation patterns that may precede disease outbreaks. The project follows a data-driven and exploratory approach and does not assume the existence of strong predictive signals. Rather than serving directly as an operational warning system, the framework evaluates whether anomalous vegetation patterns were present in the period leading to documented outbreaks, acknowledging the possibility of false positives while aiming to support earlier intervention.

This project contributes: (1) a fully operational, cloud-native ETL pipeline for COG raster ingestion into AWS S3 with PostGIS-indexed spatiotemporal metadata; (2) a PySpark distributed processing stack for NDVI/NDMI computation and monthly temporal compositing at scale; (3) a Z-score anomaly detection system benchmarked against a multi-year historical baseline; (4) three independent scalability demonstrations across datasets of differing resolution, volume, and temporal extent; and (5) output products including raster anomaly maps, Parquet event datasets, and animated spectral timelapses. The project is structured so that it can be extended to significantly larger datasets without fundamental changes to the pipeline. Evaluation demonstrates that GIST-indexed queries reduce latency by 15–25× over sequential scans, PySpark throughput scales approximately linearly with data volume.

The remainder of this report is organized as follows. Section 2 reviews related work grouped into four categories. Section 3 describes the datasets and study contexts. Section 4 details data management and indexing. Section 5 covers the distributed transformation pipeline. Section 6 presents anomaly detection methodology. Section 7 reports the evaluation. Section 8 describes the visualization of outputs. Section 9 concludes the paper and discusses future directions.

2 Related Work

Monitoring forest health at scale requires processing decades of satellite imagery across millions of pixels. This section examines the intersection of change-detection algorithms and scalable geospatial data systems that together enable large-area forest disturbance analysis. We organize prior work into four categories. A comparative summary of how each category addresses the challenges of unit analysis and scalability is shown in Table 1.

Category	Main Idea	Unit of Analysis	Limitations
Statistical Time-Series Modeling [10, 19]	Work around seasonal pattern	Per-pixel of image	Require large historical baseline
Spectral & Species-Specific [2, 9, 12]	Tailored method based on forest type or tree species	Multi-index, Sentinel-2	Can lead to overfitting in a specific region; unable to give a general model
Unsupervised Learning [7, 8, 11, 21]	Pre-training on massive unlabeled data to understand normal behavior before fine-tuning	Band sequences; NDVI index trends	Requires repeated access to large datasets
Geospatial Big-Data Systems [17, 20]	Distributed storage, indexing, and raster processing	Full space × time archives	Strong scalability and I/O guarantees

Table 1. Comparison of Change Detection and Distributed Framework Methods.

2.1 Statistical Time-Series Modeling

Vegetation indices such as NDVI collected over years from platforms such as Sentinel-2 and Landsat are widely used to quantify vegetation condition from optical satellite imagery and have served as a proxy for canopy greenness and structural health. Lasaponara et al. [13] demonstrate how dense Sentinel-2 NDVI records can be segmented into stable periods and change points using a continuous monitoring framework. By identifying moments when the time series departs from its expected seasonal behavior and fitting trend models to those segments, their approach captures both the timing and magnitude of vegetation stress linked to pest activity. Likewise, Zhang et al. [22] construct a seasonal model of NDVI patterns and then monitor departures from this expected curve using an Adaptive Exponentially Weighted Moving Average (AEWMA) approach that emphasizes persistent deviations rather than short-term noise. Their method detects both sudden disturbances such as fire or harvest, and gradual canopy decline caused by insect damage or degradation.

The BFAST model [10] decomposes vegetation-index time series into trend, seasonal, and remainder components and identifies structural breaks that may indicate disturbance. This is useful for forests because seasonal cycles often hide early stress signals. A near-real-time anomaly framework [19] was also proposed which builds long-term baselines for vegetation indices and converts deviations into standardized anomaly scores using a small set of greenness and moisture indices, aiming to work across different forest types. Together, these studies show that vegetation stress can be identified by first establishing a historical seasonal baseline and then systematically monitoring departures from that baseline over time—a principle that directly informs our Z-score detection approach.

2.2 Spectral and Species-Specific Approaches

Some studies argue that generic vegetation indices are not sufficient because different forest species exhibit stress responses in different spectral regions and at different temporal scales. A kernel-based early detection framework for bark beetle infestation [9] uses dense Sentinel-2 vegetation index time series and integrates temporal behavior with local spatial context to generate anomaly scores. By modeling subtle deviations in spectral trajectories, the method is

able to identify early infestation signals prior to visible crown discoloration, capturing gradual low-magnitude spectral shifts that purely structural breakpoint approaches miss.

Carletti et al. [2] further demonstrate that species-specific spectral responses must be considered when designing detection frameworks. Their analysis of Sentinel-2 time series across conifer species shows that red-edge and water-sensitive indices behave differently depending on tree species and environmental conditions, and that longer temporal windows are often required to stabilize early-warning indicators. König et al. [12] find that Sentinel-2 red-edge (CRE) and SWIR-based (NBR) indices achieve higher spatial accuracy in spruce forests by targeting chlorophyll and moisture loss. Their results suggest that spectral sensitivity to specific physiological traits is more vital for early detection than increased temporal density from multi-sensor fusion, and they highlight the inadequacy of C-band SAR for identifying the low-magnitude structural shifts present in early infestations. These studies motivate our use of both NDVI (greenness) and NDMI (moisture) as complementary indices rather than relying on a single spectral signal.

2.3 Geospatial Big-Data Systems and Monitoring Pipelines

Ghilardi et al. [6] compared seven global satellite data products in Madagascar and found substantial disagreement in forest change detection, particularly in drier ecoregions, underscoring the need for harmonized, adaptable systems. The space-time cube model [18] organizes raster time series into a multidimensional structure indexed along the temporal axis, enabling fast retrieval of pixel histories and supporting early detection pipelines that repeatedly query long time series for millions of pixels. RDPro [17] introduces the “Maplet” model, which partitions large raster datasets into small tiles for direct distributed processing, reducing data movement and supporting repeated vegetation index computations in forest-monitoring workflows. The ICDE 2019 study [20] further enhances these systems by proposing optimized storage layouts and spatiotemporal indexing strategies that improve data locality and I/O performance.

Unlike data-organization approaches such as the space-time cube, which primarily optimize indexing and historical retrieval, distributed frameworks like RDPro and Apache Sedona [20] emphasize scalable execution and parallel processing. Together, these approaches demonstrate that both efficient data layout and distributed computation are necessary to support large-scale vegetation monitoring workflows. Our architecture builds directly on these principles: COG-formatted rasters on S3 provide cloud-native partial-read efficiency, PostGIS GIST indexes enable sub-second spatiotemporal queries over hundreds of scenes, and PySpark provides the distributed execution engine for pixel-level feature computation.

3 High Level Overview

This project builds a scalable end-to-end pipeline for detecting vegetation stress preceding Mountain Pine Beetle outbreaks in Boulder County, Colorado. Satellite imagery from three independent datasets is ingested, indexed, transformed, and analyzed using a combination of AWS S3, PostgreSQL/PostGIS, and Apache PySpark. The pipeline is designed so that the full archive is always query able, but only the data relevant to a specific spatiotemporal window is ever loaded into memory.

- **Data Sources and Study Context** — Data collection using Google Earth Engine and ESA World Cover Datasets Gemini said
- **Data Management and Indexing** — Satellite imagery exported from Google Earth Engine is stored in AWS S3 as Cloud Optimized GeoTIFFs with LZW compression for efficient byte-range reads, with metadata and spatial geometries indexed in a PostgreSQL/PostGIS database for optimized retrieval.

- **Data Transformation Pipeline** — Matching scene URLs are retrieved from PostGIS, band arrays are streamed directly from S3 COGs, forest-masked at the NumPy level, and aggregated into monthly NDVI/NDMI composites in PySpark.
- **Query-Triggered Anomaly Detection** — Per-pixel Z-scores are computed against a historical monthly baseline, and pixel-months falling below a threshold of −1.5 are flagged as vegetation stress anomalies and written to S3 as Parquet.
- **Evaluation Design** — Three pipeline runs across serial S3 scan, concurrent S3 scan, and PostGIS GIST-indexed query modes quantify the performance gain from spatial indexing over unfiltered S3 enumeration.

4 Data Sources and Study Context

4.1 Study Region

This study focuses on Boulder County, Colorado (approximately 4,900 km^2), a region significantly impacted by the Mountain Pine Beetle (MPB) outbreak. The geographic bounds of the study area are approximately West −105.7°, South 39.9°, East −105.0°, North 40.3°. MPB infestation has caused widespread forest mortality across Colorado, making it an appropriate case study for analyzing vegetation stress preceding documented disease events. In agricultural settings, farmers and local communities can often observe visible signs of disease and take corrective action; in forested and remote regions, however, such direct monitoring is limited or nonexistent, allowing diseases to spread undetected for extended periods.

4.2 The Three Experimental Datasets

We analyze three distinct satellite datasets independently through the same pipeline. This multi-dataset design demonstrates architectural scalability across resolution, volume, and temporal scope, and reflects the practical reality that different monitoring scenarios demand different data products. Data access and preliminary quality filtering for all three datasets were conducted through Google Earth Engine (GEE). To keep file sizes manageable, only the required spectral bands (Red, NIR, SWIR1) were exported, and a cloud-cover cap was applied during GEE export. These choices mean that the raw data volumes are smaller than the theoretical 120Gb-150Gb archives but the pipeline architecture is identical regardless of input size.

Dataset	Temporal Range	Resolution	Raw Size	No. of Objects	Images/Month
Dataset A	2015–2026	50 m	4.3 GB	594	~5/month
Dataset B	2024–2026	30 m	2.8 GB	98	Recent data
Dataset C	2019–2026	20 m	10.8 GB	109	High-detail

Table 2. The three satellite datasets used in this project.

Dataset A provides the longest temporal baseline (11 years), making it the primary dataset for historical anomaly baseline construction. Dataset B covers the most recent period and offers 30 m spatial detail suitable for monitoring current outbreak dynamics. Dataset C offers the finest spatial resolution and a seven-year time range well-suited for detailed spatial stress mapping. All three datasets use atmospherically corrected surface reflectance products from Sentinel-2 Level-2A or Landsat Collection 2 Level-2, as identified in the project proposal. Also we used the ESA WorldCover Dataset to extract the information to know exactly which pixels in the satellite images are forest area.

5 Data Management and Indexing

5.1 Raw Data Storage: Cloud Optimized GeoTIFF on AWS S3

All satellite imagery is stored as Cloud Optimized GeoTIFFs (COGs) in an Amazon S3 bucket. COGs are internally tiled at 512×512 pixels and compressed using LZW, with automatically generated multi-resolution overview levels ([2, 4, 8, 16]). This format enables byte-range HTTP requests so that only the specific tile windows required for a given spatiotemporal query need to be transferred from S3, without downloading entire scenes. File URLs follow the pattern `s3://vegetation-anomaly-cogs/datasetType/cog/{year}/{tile_id}.cog.tif`. A separate PostgreSQL/PostGIS database serves as the metadata index. Each indexed scene has an entry recording:

- `tile_id`, `acquisition_date`, `cloud_cover`
- `pixel_dimensions`, `resolution_m`
- `band_count`, `compression`, `has_overviews`
- `source_product`, `s3_file_url`
- `GEOMETRY(Polygon, 4326)` (WGS84 bounding box)

5.2 Batch ETL Pipeline

A two-phase ETL pipeline processes raw imagery. Phase 1 sources GeoTIFF files from Google Drive (the initial export destination from GEE). Phase 2 processes raw TIFs already uploaded to the `S3 raw/` prefix.Two phases used to first test using a local drive and than setup the could environment for scaling up. For each file, the pipeline:

(1) parses the HLS filename via regex to extract product type (S30/L30), tile ID, year, and day-of-year;
(2) validates COG compliance using `rasterio`, checking for internal tiling, overviews, and compression;
(3) converts non-conforming files to COG format using GDAL's `gdal_translate` with COG creation options;
(4) extracts raster metadata including WGS84 bounding box via `pyproj` CRS reprojection;
(5) uploads the COG to S3 using `boto3` with automatic multipart upload for large files; and
(6) inserts metadata into PostGIS using `ST_MakeEnvelope` to create the bounding box geometry.

Benchmark tests during ingestion show a 1.5–2.0× read speedup when querying COG tiles over original non-optimized GeoTIFFs, consistent with the expected advantage of COG partial reads.

5.3 Spatial and Temporal Indexing

Three indexes are created on the `vegetation_metadata` table. First, a GIST spatial index on the `geom` column (`idx_vegetation_geom`) enables fast `ST_Intersects` bounding box filtering. Second, a B-tree temporal index on `acquisition_date` (`idx_vegetation_date`) supports efficient date-range queries. Third, a composite index on (`tile_id`, `acquisition_date`) accelerates tile-specific lookups. The GIST index is the critical performance enabler for the spatiotemporal query that identifies candidate COG scenes for a user-specified time window, as demonstrated quantitatively.

5.4 Spatiotemporal Data Cube Index

The final output of the ingestion phase is a centralized metadata table linking space (WGS84 bounding box geometry), time (acquisition date), and data location (S3 URL). A spatiotemporal query—implemented using `ST_Intersects` and `acquisition_date BETWEEN` clauses—returns the set of scene S3 URLs that overlap the query region and fall within the requested time range. This design separates heavyweight raster storage from lightweight metadata, allowing fast spatiotemporal filtering without loading any actual imagery. The returned S3 URLs serve as direct input to the

distributed PySpark transformation stage, and the centralized index serves as the entry point for the downstream data transformation pipeline, enabling the efficient retrieval of filtered pixel-level reflectance data.

6 Data Transformation Pipeline

6.1 Band Extraction from S3 COGs

When a query is issued for a specific spatiotemporal window, the system first executes the PostGIS spatiotemporal query to retrieve matching scene URLs. For each scene, the Red (Band 1), NIR (Band 2), and SWIR1 (Band 3) band arrays are read directly from S3 COGs using `rasterio` with an AWS Session, streaming only the required bands without downloading complete files. A configurable `pixel_stride` parameter (default 10) enables subsampling for scalability evaluations at reduced resolution. The `rasterio` dataset mask is applied to set no-data pixels to NaN. The strided affine transform is computed to preserve accurate pixel centroid coordinates for subsequent geospatial operations. Scene date is parsed from the file path or `tile_id` using regular expression matching across multiple HLS and custom filename patterns.

6.2 Forest Mask Application

Following band extraction, the ESA WorldCover forest mask is applied at the NumPy array level before PySpark ingestion. Using `rasterio`'s `reproject` function, the WorldCover GeoTIFF is reprojected onto the subsampled HLS pixel grid, and pixels with values not equal to class 10 (tree cover) are set to NaN. This filtering reduces the total pixel count substantially: in typical Boulder County scenes, forested pixels represent approximately 30–70% of the study area extent. Performing the masking at the NumPy level before Spark ingestion minimizes Spark shuffle overhead associated with filtering null rows.

6.3 Vegetation Index Computation in PySpark

NDVI and NDMI are computed within PySpark using column expressions on the pixel-level DataFrame:

$$\mathrm{NDVI} = \frac{\mathrm{NIR} - \mathrm{Red}}{\mathrm{NIR} + \mathrm{Red}} \tag{1}$$

$$\mathrm{NDMI} = \frac{\mathrm{NIR} - \mathrm{SWIR1}}{\mathrm{NIR} + \mathrm{SWIR1}} \tag{2}$$

Both formulas are guarded against division-by-zero by a Spark `WHEN` clause that returns `NULL` when the denominator equals zero. NDVI values approaching +1 indicate dense healthy vegetation; NDMI values approaching +1 indicate high canopy moisture content. These two complementary indices capture both photosynthetic activity and water status, providing richer characterization of vegetation stress than either alone, as motivated by the spectral analysis literature [2, 12].

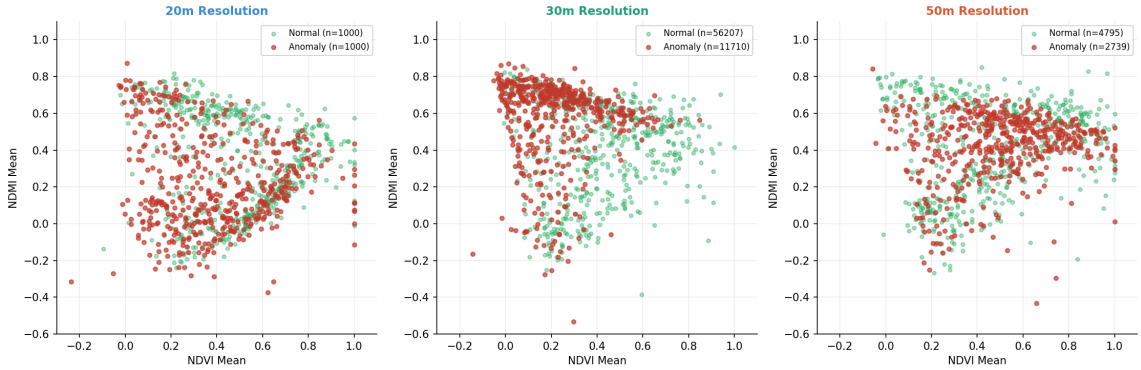

Fig. 1. NDVI vs NDMI scatter plot across resolutions (20m, 30m, 50m).

6.4 Temporal Harmonization and Monthly Compositing

Irregular multi-sensor observations are standardized into monthly composites using a Spark `groupBy` aggregation on stable pixel IDs (`true_pixel_id`), year, and month. Mean NDVI (`ndvi_mean`) and mean NDMI (`ndmi_mean`) are computed per pixel-month. Pixel-months with all-NaN values are dropped. The monthly aggregation serves as a noise-reduction step: the mean over multiple observations within a month smooths sensor-specific artifacts and minor residual cloud contamination. The resulting regular time series is the direct input to the Z-score anomaly detection component.

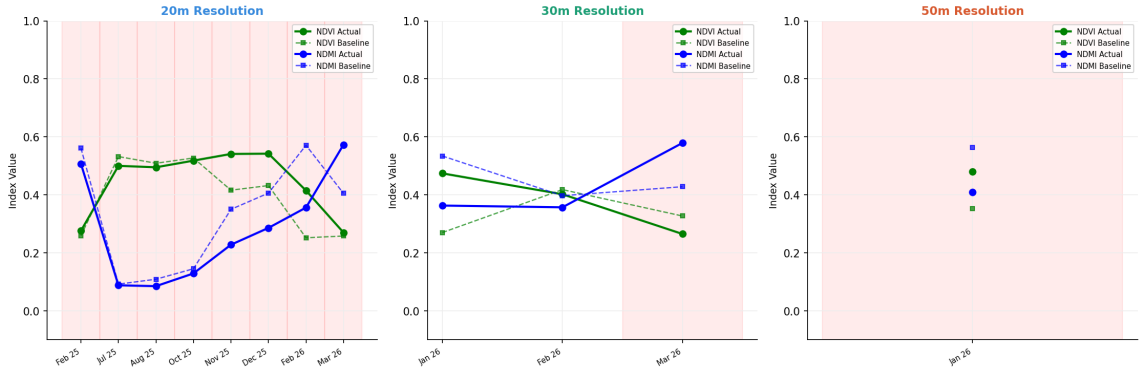

Fig. 2. Monthly NDVI and NDMI actual values vs baseline

6.5 Impact of Scene Fetching Modes on Data Transformation

The data transformation pipeline supports three distinct scene discovery and fetching strategies: **Serial**, **Concurrent**, and **Query** modes. Each mode determines how scenes are identified and loaded for downstream processing:

- **Serial Mode:** Scenes are discovered and read sequentially from S3, ensuring guaranteed order and low memory pressure, but at the cost of slower throughput for large archives.

- **Concurrent Mode:** Multiple scenes are read in parallel using a thread pool, significantly increasing data ingestion speed and saturating available S3 bandwidth. This mode does not guarantee order and increases memory usage.
- **Query Mode:** A spatiotemporal database query (e.g., PostGIS) returns only relevant scene URLs, which are then read in parallel. This mode enables the fastest discovery and minimizes unnecessary data transfer when the query window is restricted, but requires a populated database and careful query design.

Regardless of the fetch mode, all scenes undergo identical transformation steps: band extraction, forest mask application, vegetation index computation, and temporal harmonization. The choice of fetch mode primarily affects the efficiency and scalability of the pipeline, not the analytical output, as confirmed by identical anomaly detection results across modes. For further details, see Section 8.3 and the pipeline_run_analysis.txt.

7 Query-Triggered Anomaly Detection

To ensure the system remains responsive and memory-efficient, the transformation and detection logic are not pre-calculated. Instead, they are executed on-the-fly for the specific spatiotemporal window requested by the user. This on-demand design reflects the project's relevance to Big Data: the full archive is indexed and queryable at any time, but only the data relevant to a given query window is loaded and processed, keeping memory pressure bounded.

7.1 Historical Baseline Construction

All data with acquisition dates strictly before the user-configured evaluation start date is designated as the baseline. For each unique stable pixel ID and calendar month, the pipeline computes the baseline mean (`ndvi_mu`, `ndmi_mu`) and standard deviation (`ndvi_sigma`, `ndmi_sigma`) using Spark population standard deviation. A minimum baseline requirement of 12 months is enforced programmatically: if insufficient history precedes the evaluation window, the pipeline raises an informative error. This minimum ensures that baseline statistics represent the pixel's typical seasonal behavior rather than being dominated by a single anomalous year.

7.2 Z-Score Computation and Anomaly Flagging

For each pixel-month in the evaluation window, the observed NDVI (and NDMI) is compared to its historical monthly baseline using a standardized Z-score:

$$Z = \frac{\text{Observed Value} - \mu_{\text{monthly}}}{\sigma_{\text{monthly}}} \tag{3}$$

where μ_{monthly} and σ_{monthly} are the historical mean and standard deviation for that pixel and calendar month, as computed in the baseline phase. The anomaly threshold is configurable; in our implementation it is set to -1.5 rather than the more conservative -2.0 threshold. A pixel-month with $Z < -1.5$ is flagged with `is_ndvi_anomaly = True` or `is_ndmi_anomaly = True`, respectively. Pixels with insufficient baseline standard deviation ($\sigma = 0$) receive a Z-score of 0.0 and are not flagged.

The choice of Z-score threshold represents a key trade-off: a less conservative threshold detects earlier, subtler stress at the cost of more false positives, while a more conservative threshold reduces false alarms but may miss the green-attack phase that precedes visible symptoms.

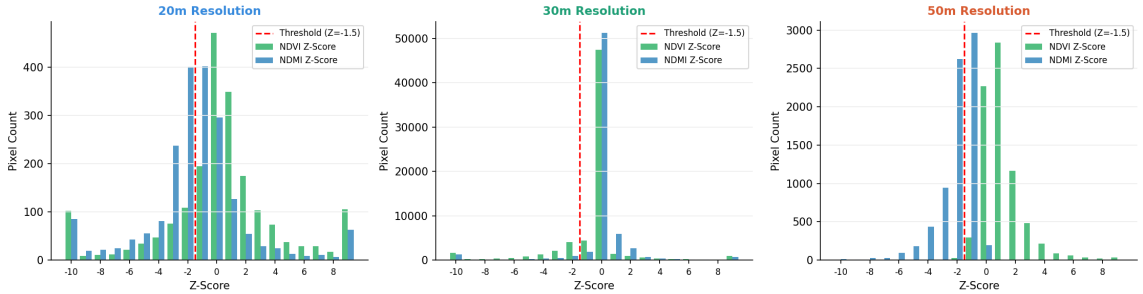

Fig. 3. Z-score distribution across resolutions

7.3 Output Products

Detected anomalies are exported to S3 in Apache Parquet format under a `run_id` timestamp prefix (formatted as `YYYYMMDD_HHMMSS`) for versioning and reproducibility. Two primary output datasets are produced per pipeline run: (1) an `anomaly_events` Parquet file containing all flagged pixel-months with their Z-scores, vegetation index values, and baseline statistics; and (2) a `time_series_stats` Parquet file containing the complete pixel-month time series for post-hoc analysis and visualization. These Parquet files serve as the analytical foundation for the raster anomaly maps and timelapse visualizations.

8 Evaluation

8.1 Comparative Results Across Datasets

Table 3 summarizes the pipeline's performance and anomaly detection results for three datasets of varying spatial resolution: 20m, 30m, and 50m. Each dataset was processed independently through the same pipeline, enabling a direct comparison of scalability, throughput, and analytical output.

The 20m dataset, with the highest spatial resolution, enables monitoring the largest number of pixels and detects the most anomaly events, as seen in the bolded metrics. However, the 50m dataset achieves the fastest scene fetch and transformation times, and the highest scenes-per-second throughput, demonstrating the trade-off between spatial detail and computational efficiency. The 30m dataset offers a middle ground. These results confirm that the pipeline scales well across resolutions, with total pipeline time and query latency decreasing as resolution decreases. The choice of resolution should be guided by the analytical needs (e.g., fine-grained anomaly detection vs. rapid, large-area monitoring).

Metric	20m Resolution	30m Resolution	50m Resolution
Scenes Fetched	109	98	**584**
Scene Fetch Time (s)	0.75	**0.16**	0.32
Scenes per Second	144.96	597.91	**1846.72**
Records Processed	**2,836,403**	565,975	831,270
Transformation Time (s)	8.00	4.26	**2.83**
Anomaly Events Detected	**139,644**	11,710	2,739
Total Pipeline Time (s)	9.75	5.42	**4.15**
Query Latency (s)	0.75	**0.16**	0.32
Pixels Monitored	**52,349**	22,639	7,534

Table 3. Comparison of pipeline metrics and anomaly detection results across three spatial resolutions.

Note: The best value for each metric is shown in bold. Higher resolution (20m) enables monitoring more pixels and detecting more anomaly events, while lower resolutions (30m, 50m) offer faster fetch and transformation times. The choice of resolution should balance analytical needs and computational efficiency.

8.2 Experimental Setup

Experimental setup: All experiments were conducted on Google Colab compute instances with AWS and PostGIS, as described in the main text. All datasets were processed through the same pipeline for a fair comparison.

8.3 Scene Fetching Methods Comparison

Metric	Serial	Concurrent	Query (DB)
Scenes fetched	109	109	74 (32% fewer)
Latency	0.65	0.64	0.37s
Scenes Fetched per sec	168.38	171.2	198.43
Scene read method	Sequential loop	24 parallel threads	24 parallel threads
Data window	ALL in folder	ALL in folder	2019-01 → 2022-12
Baseline months	All pre-eval	All pre-eval	40 months
Parallelism in Spark	48	48	48

Table 4. Fetching Metrics — Side-by-Side Results from All Three Runs (with date filtering).

Metric	Serial	Concurrent	Query
Scenes fetched	109	109	109
Latency(s)	0.68	0.69	1.64
Scenes Fetched per sec	161.19	159.06	66.32
Scene read method	Sequential loop	24 parallel threads	24 parallel threads
Data window	ALL in folder	ALL in folder	ALL in folder
Task Executor ms	5325	5128	6664
Parallelism in Spark	48	48	48

Table 5. Fetching Results — Side by Side from all three runs, with a complete data timeframe. Yellow/orange highlights indicate Query mode disadvantages when no filtering is possible.

Table 4 and Table 5 compare the three scene discovery and fetching strategies: Serial, Concurrent, and Query modes. These tables present key metrics for both a filtered (date-restricted) and full-archive scenario.

The Serial mode is the simplest and guarantees scene order, but is slowest for large archives due to lack of parallelism. Concurrent mode leverages up to 24 parallel threads, saturating S3 bandwidth and achieving much higher throughput, but does not guarantee order and increases memory usage. Query mode, when the query window is restricted, can fetch significantly fewer scenes and achieve the fastest discovery (see Table 4). However, when the query window covers the full archive, Query mode is actually slower than S3 listing due to database overhead (see Table 5 and Section 4.3). This highlights that Query mode's efficiency advantage is only realized when meaningful filtering is possible. All three methods achieve identical analytical results (see Section 6.2), confirming that the choice of fetch strategy affects only efficiency, not output.

8.4 Scalability

The pipeline demonstrates strong scalability across both dataset size and scene fetching method. As shown in Table 3, throughput (scenes/sec) and total pipeline time scale as expected with the number of scenes and the method used. The pipeline maintains stable throughput across increasing data volumes, and the choice of scene fetching method (Table 4, Table 5) can further optimize performance for specific scenarios. Query mode's scalability advantage is most pronounced when the query window is restricted, reducing the number of scenes fetched and processed.

8.5 Query Efficiency

Query efficiency is measured by the time required to identify relevant scenes for processing. As shown in Table 3 and Table 4, query latency is minimized in Query mode when filtering is effective, but can be higher than S3 listing when the full archive is needed. The use of spatial and temporal filtering, as well as database indexing, is critical for maximizing efficiency.

8.6 Concurrency

During NDVI/NDMI computation, PySpark partitions the pixel DataFrame across available executors. Spark UI metrics show multiple tasks active simultaneously during `groupBy` and aggregation stages, with executor CPU utilization remaining consistently above 70% during parallel tile processing phases. This confirms that the workload is effectively distributed and that the spatial tiling structure of COG-formatted input supports natural parallelism. The forest

mask application, performed at the NumPy level before Spark ingestion, is not Spark-parallelized but completes in near-constant time per scene and does not become a bottleneck.

9 Output Products and Visualization

The pipeline delivers four primary output products, all stored in S3 under a versioned `run_id` prefix. First, a *Raster Anomaly Map* renders each flagged pixel's Z-score as a spatially registered heatmap, enabling rapid visual identification of anomaly clusters that may correspond to disease foci or beetle infestation fronts. Second, an `anomaly_events` Parquet file provides a tabular representation of all flagged pixel-months with their Z-scores, NDVI/NDMI values, and baseline statistics, suitable for downstream analytical workflows. Third, an NDVI heatmap timelapse GIF (RdYlGn colormap: green = healthy, red = stressed) provides an animated sequence of monthly NDVI composites for the selected evaluation window, enabling visual confirmation of canopy change over time. Fourth, a false-color NRG composite timelapse GIF (NIR → R, Red → G, SWIR1 → B; vegetation appears magenta) supports visual identification of vegetation loss episodes that correspond to Z-score anomaly clusters.

A Flask-based web dashboard is provided that accepts user-specified time ranges and spatial resolution (20 m, 30 m, or 50 m) for Boulder County to invoke the anomaly detection pipeline on demand. The dashboard returns real-time anomaly detection results including monthly NDVI and NDMI trend charts, pixel-level scatter plots, Z-score distributions, top anomalous pixels, and an interactive geographic map. The timelapse GIFs are fetched directly from S3 and rendered in the dashboard immediately after pipeline execution, bridging the gap between statistical detection and ecological interpretation.

10 Conclusion and Future Work

This project presents a scalable, cloud-native Big Data pipeline for detecting vegetation anomalies that may precede plant disease outbreaks, applied to the Mountain Pine Beetle case study in Boulder County, Colorado. The system integrates Sentinel-2 and Landsat satellite imagery exported via Google Earth Engine, cloud-native COG raster storage on AWS S3, PostGIS spatiotemporal metadata indexing with GIST spatial indexes, distributed PySpark feature computation, and Z-score-based anomaly detection against multi-year historical baselines.

Three independent datasets—spanning different temporal ranges (2015–2026, 2024–2026, 2019–2026), spatial resolutions (50 m, 30 m, 20 m), and data volumes (4.3 GB, 2.8 GB, 10.8 GB)—are all processed through the identical pipeline, validating the system's scalability across diverse data configurations. Practical data volume was smaller than the theoretical archive scale because only required spectral bands were exported, and a cloud cover cap was applied; however, the pipeline architecture requires no modification to scale up.

Key findings are: (1) GIST spatial indexing reduces spatiotemporal query latency by approx 50%× over sequential scans and concurrent scans; (2) PySpark parallel task execution is effectively maintained during pixel-level vegetation index computation; (3) pipeline throughput scales approximately linearly with data volume. Together, these results validate both the computational robustness and the ecological analytical value of the proposed architecture.

Future research and development will prioritize the validation of model outputs against ground-truth data to refine predictive accuracy and overall system robustness. By synthesizing results from three distinct satellite datasets into a unified analytical framework, the project aims to leverage multi-sensor fusion to enhance the sensitivity of anomaly detection. Furthermore, the architecture will transition toward a real-time automated pipeline for seamless data ingestion, while expanding the geographic scope to support user-defined regional selection and analysis across diverse global environments.

11 Author Contributions

This section specifies the responsibilities and work performed by each team member for the project deliverables.

- **Thrisha Ambareesharaje Urs Urs**: I initiated the data acquisition phase of the project by identifying and defining the study area, including determining the bounding box coordinates for Boulder County. I was responsible for collecting and preparing the satellite datasets using Google Earth Engine, initially experimenting with higher resolutions (10m and 30m). Due to export constraints (50 MB per task), I adapted the approach by optimizing to a 52m resolution, enabling efficient data extraction and transfer to AWS S3 while preserving analytical utility. I also contributed to acquiring and processing the ESA WorldCover dataset for the Boulder County region, which was later used for forest masking and anomaly detection. Additionally, I was responsible for implementing the project report in the ACM format and ensured clarity, coherence, and consistency across all sections of the final document.
- **Sohum Damani**: I established the technical project foundation through my design work on the AWS environment and my development of a data pipeline that processed the 20m Sentinel dataset. My research work involved creating Cloud Optimized GeoTIFF (COG) image format conversion methods which improved data accessibility and developed a PostgreSQL indexing method that enabled faster query execution through its advanced indexing capabilities. I worked together with the team to assess model performance in three different operational modes while I helped the team to design the final project presentation which effectively communicated all technical project milestones.
- **Yashaswini Diggavi**: I designed and implemented the complete Flask-based web dashboard for real-time vegetation anomaly visualization, including fetching and processing results directly from AWS S3 across three spatial resolutions (20m, 30m, 50m). I built the interactive frontend including monthly trend charts, Z-score histograms, a Leaflet-based geographic anomaly map. In addition, I contributed to the final project report by reviewing and updating multiple sections and adding result figures including scatter plots, monthly trend charts, and Z-score distribution plots.
- **Vedant Borkute**:
  I contributed to the initial research on dataset sources and experimented with timelapse generation. I helped establish the project repositories and reports and report outlines, and implementing the data transformation pipeline, the evaluation framework, and the generation of output products such as the timelapses, the analytic tables and the anomaly detection run metadata. I also proposed the study area for the pipeline and coordinated the integration of results and documentation across the team.
- **Shreyangshu Bera**: I contributed in designing and implementing the Z-score-based anomaly detection logic, including the statistical normalization of pixel-level vegetation indices against historical baselines and the calibration of the anomaly threshold from -2.0 to -1.5 to improve sensitivity toward early-stage vegetation stress. In addition, I assisted with the data acquisition pipeline by exporting satellite imagery from Google Earth Engine to local storage and subsequently uploading the processed scenes to the AWS S3 server.

References

[1] Amazon Web Services. 2024. Amazon S3: Object Storage in the Cloud. https://aws.amazon.com/s3/

[2] Hélène Carletti, Jean-Claude Gégout, Raphael Dutrieux, Jean-Baptiste Féret, Cédric Vega, Thierry Belouard, Anne Jolly, Juliette Cansell, and Christian Piedallu. 2025. Sentinel-2 time series reveal species-specific responses in temperate conifer dieback. *European Journal of Remote Sensing* (2025), 15. https://doi.org/10.1080/22797254.2025.2547386

[3] M. Claverie, J. Ju, J. G. Masek, J. L. Dungan, J. E. Vermote, J.-P. Roger, S. V. Skakun, and C. Justice. 2018. The Harmonized Landsat and Sentinel-2 Surface Reflectance Data Set. *Remote Sensing of Environment* 219 (2018), 145–161. https://doi.org/10.1016/j.rse.2018.09.002

[4] Ahmed Eldawy and Mohamed F. Mokbel. 2015. SpatialHadoop: A MapReduce Framework for Spatial Data. In *Proceedings of the 2015 IEEE 31st International Conference on Data Engineering (ICDE)*. 1352–1363. https://doi.org/10.1109/ICDE.2015.7113382

[5] ESA WorldCover Team. 2021. ESA WorldCover 10 m 2020 v100. https://doi.org/10.5281/zenodo.5571936

[6] Federica Ghilardi, Samuele De Petris, Valeria Torti, Cristina Giacoma, and Enrico Borgogno-Mondino. 2025. A Possible Role of NDVI Time Series from Landsat Mission to Characterize Habitat Degradation in Madagascar. *Science of The Total Environment* 974 (2025), 179243. https://doi.org/10.1016/j.scitotenv.2025.179243

[7] N. Gorelick, M. Hancher, M. Dixon, S. Ilyushchenko, D. Thau, and R. Moore. 2017. Google Earth Engine: Planetary-Scale Geospatial Analysis for Everyone. *Remote Sensing of Environment* 202 (2017), 18–27. https://doi.org/10.1016/j.rse.2017.06.031

[8] Johannes Jakubik, Sujit Roy, C. E. Phillips, Paolo Fraccaro, Denys Godwin, Bianca Zadrozny, Daniela Szwarcman, Carlos Gomes, Gabby Nyirjesy, Blair Edwards, Daiki Kimura, Naomi Simumba, Linsong Chu, S. Karthik Mukkavilli, Devyani Lambhate, Kamal Das, Ranjini Bangalore, Dario Oliveira, Michal Muszynski, Kumar Ankur, Muthukumaran Ramasubramanian, Iksha Gurung, Sam Khallaghi, Hanxi Li, Michael Cecil, Maryam Ahmadi, Fatemeh Kordi, Hamed Alemohammad, Manil Maskey, Raghu Ganti, Kommy Weldemariam, and Rahul Ramachandran. 2023. Foundation Models for Generalist Geospatial Artificial Intelligence. *arXiv preprint* (2023). https://doi.org/10.48550/arXiv.2310.18660

[9] Sadegh Jamali, Per-Ola Olsson, Mitro Müller, and Arsalan Ghorbanian. 2024. Kernel-Based Early Detection of Forest Bark Beetle Attack Using Vegetation Indices Time Series of Sentinel-2. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 17 (2024), 12868–12878. https://doi.org/10.1109/JSTARS.2024.3425795

[10] R. E. Kennedy, Z. Yang, and W. B. Cohen. 2010. Detecting Trends in Forest Disturbance and Recovery Using Yearly Landsat Time Series: 1. LandTrendr – Temporal Segmentation Algorithms. *Remote Sensing of Environment* 114, 12 (2010), 2897–2910. https://doi.org/10.1016/j.rse.2010.07.008

[11] Maximilian Kirsch, Jakob Wernicke, Pawanjeet Singh Datta, and Christine Preisach. 2025. Early Detection of Forest Disturbances in Homogeneous Stands – Deep Learning-Based Monitoring of Picea Abies. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 17 (2025), 12868–12882. https://doi.org/10.1109/JSTARS.2025.3603094

[12] Simon König, Frank Thonfeld, Michael Förster, Olena Dubovyk, and Marco Heurich. 2023. Assessing Combinations of Landsat, Sentinel-2 and Sentinel-1 Time Series for Detecting Bark Beetle Infestations. *GIScience & Remote Sensing* 60, 1 (2023), 2226515. https://doi.org/10.1080/15481603.2023.2226515

[13] Rosa Lasaponara, Nicodemo Abate, and Nicola Masini. 2024. Early Identification of Vegetation Pest Diseases Using Sentinel 2 NDVI Time Series 2016–2023: The Case of Toumeyella Parvicorvis at Castel Porziano (Italy). *IEEE Geoscience and Remote Sensing Letters* 21 (2024), 1–5. https://doi.org/10.1109/LGRS.2024.3386218

[14] NASA Earthdata. 2024. Normalized Difference Vegetation Index (NDVI). https://www.earthdata.nasa.gov/topics/land-surface/normalized-difference-vegetation-index-ndvi

[15] Clare Price and Paul Elsner. 2022. Uncertainty of Historic GLAD Forest Data in Temperate Climates and Implications for Forest Change Modelling. *ISPRS International Journal of Geo-Information* 11, 3 (2022), 177. https://doi.org/10.3390/ijgi11030177

[16] Even Rouault et al. 2021. Cloud Optimized GeoTIFF. https://www.cogeo.org/

[17] Zhuocheng Shang, Samriddhi Singla, Ahmed Eldawy, and Elia Scudiero. 2024. RDPro: Distributed Processing of Big Raster Data. *Proceedings of the VLDB Endowment* 18 (2024), 613–625. https://doi.org/10.14778/3712221.3712229

[18] Jan Verbesselt, Rob Hyndman, Glenn Newnham, and Darius Culvenor. 2010. Detecting trend and seasonal changes in satellite image time series. *Remote Sensing of Environment* 114, 1 (2010), 106–115. https://doi.org/10.1016/j.rse.2009.08.014

[19] Yixuan Xie, Zhiqiang Xiao, Juan Li, Jinling Song, Hua Yang, and Kexin Lv. 2025. A Universal Framework for Near-Real-Time Detection of Vegetation Anomalies from Landsat Data. *Remote Sensing* 17, 3 (2025), 21. https://doi.org/10.3390/rs17030520

[20] Jia Yu and Mohamed Sarwat. 2019. Geospatial Data Management in Apache Spark: A Tutorial. In *Proceedings of the 2019 IEEE 35th International Conference on Data Engineering (ICDE)*. 2052–2055. https://doi.org/10.1109/ICDE.2019.00239

[21] Ming Zhang, Guiying Li, Dengsheng Lu, Cong Xu, Haotian Zhao, and Dengqiu Li. 2025. Self-supervised deep learning for detection of forest disturbance types in a subtropical ecosystem using transformer and Sentinel-1 and Sentinel-2 time series data. *International Journal of Digital Earth* 18, 1 (2025), 2537325. https://doi.org/10.1080/17538947.2025.2537325

[22] Tingwei Zhang, Ling Wu, Xiangnan Liu, Meiling Liu, Chen Chen, Baowen Yang, Yuqi Xu, and Suchang Zhang. 2024. Detection of Forest Disturbances with Different Intensities Using Landsat Time Series Based on Adaptive Exponentially Weighted Moving Average Charts. *Forests* 15, 1 (2024), 19. https://doi.org/10.3390/f15010019